

From Scratch:

The Design and Implementation of a SAT Solver From Scratch

Oliver Lie

June 21, 2026

1 Abstract

This paper discusses the design and implementation methods of a SAT solver. It explores... INSERT METHODS HERE

2 Introduction

SAT solvers are amazing programs with one simple purpose: solve boolean algebra equations quickly.

2.1 CNF

CNF stands for "Conjunctive Normal Form," sometimes called "Clausal Normal Form." An equation in this form is a "conjunction of disjunctions," ie, it is an equation of "ands" ($\wedge$) in between clauses of "ors" ($\vee$). Each clause of "ors" contains one or more literals, or variables. Each literal has the option to be negated ($\neg$), however, no negation can appear in front of a clause itself. Some examples of CNF equations includes:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are not CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form (read ahead)

2.2 DNF

DNF stands for "Disjunctive Normal Form." Just like in CNF form, a DNF equation has only three operators: conjunction ($\wedge$), disjunction ($\vee$), and negation ($\neg$). A DNF equation is in the form of a "disjunction of conjunctions," ie, it is an equation of "ors" ($\vee$) in between clauses of "ands" ($\wedge$). Each clause of "ands" contains one or more literals, each with the option of being negated ($\neg$), with no negation being in front of the clauses themselves. Some examples of DNF equations include:

- $(A \wedge B)$
- $(A \wedge B) \vee (C)$
- $(A \wedge B \wedge C) \vee (D \wedge E)$

Some examples that are not DNF:

- $(A \vee B)$, this is CNF form
- $\neg(A \wedge B) \vee (C)$, where there is a NOT in front of a clause
- $(A \wedge B \vee C) \vee (D \wedge E)$, where there is an OR within a clause

2.3 Defining our Input

For this SAT solver, we will use the DIMACS CNF format of input. DIMACS CNF format consists of seven rules:¹

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.

¹<https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html>

2. In some examples of CNF files, the definition of the last clause is not terminated by a final '0'
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with 'c'²

For a quick example, here is a simple way to format a DIMACS CNF equation:

```
p cnf 4 2
1 2 - 3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment 1 2 - 3 0
-1 4 0
```

2.4 Input Processing

In order to process our input, we will provide a few APIs. We will firstly, disallow the header line. Ie, if no header is passed in, rather than throwing an error, we will simply infer the number of variables and clauses. We will also be very loose in terms of what input is considered valid. Our rules will be as follows:

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with 'c'.
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with '0', and can extend multiple lines.
4. All variables are numbered 1 - N, where N is the number of variables. Ie, if you say 10 variables, they must be labeled 1, 2, ..., N. And not 2, 3, ..., 11 or any other convention.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 - 3 0
-1 4 0
```

²<https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html>

```

4 2
1 2 - 3 0
-1 4 0

```

```

p cnf 4 2
c this comment doesn't have to be at the top
1 2 - 3 0
-1 4 0

```

```

p cnf 4 2
c this comment doesn't have to be at the top
1 2 - 3 0
-1 4

```

```

c the first two numbers are still the number
c of variables and the number of clauses
p cnf 4 2 1 2 - 3 0
-1 4 0

```

Now that we have fully defined our input rules, we can now define our API.

```

class SATSolver: public interface
    SATSolver(string input);
    int getNumVars();
    int getNumClauses();
    vector<vector<int>> getClauses();
    optional<vector<bool>> getSolution();
    void printSolution();
    bool solveCNF();
class SATSolver: private interface
    void parse(string equation);
    int numVars;
    int numClauses;
    vector<vector<int>> clauses;
    optional<vector<bool>> lits;

```

Our API takes in a string, either representing a file or a CNF equation and parses it into memory for our usage.

Pseudocode:

```

SATSolver(string input):
    lits ← no-value
    if (input is file) then
        string content ← readFile(input)
        parse(content)
    else
        parse(input)
void parse(equation):
    if (contains(equation, "p cnf")) then
        int[0...n] A
        for each line in input:

```

```

        if (startsWith(line, 'c')) then
            skip to next line
        else
            for each token in line:
                A ← A::token
numVars ← A[0]
numClauses ← A[1]
idx ← 2
for i ← 0 to numClauses:
    int[0..n] clause
    while A[idx] ≠ 0
        clause ← A[idx]
        idx ← idx + 1
    clauses ← clause
else
    numVars ← 0
    numClauses ← 0
    int[0..n] A
    for each line in input:
        if (startsWith(line, 'c')) then
            skip to next line
        else
            for each token in line:
                numVars ← max(token, numVars)
                if token = 0 then
                    numClauses ← numClauses + 1
                A ← A::token
    idx ← 0
    for i ← 0 to numClauses:
        int[0..n] clause
        while A[idx] ≠ 0
            clause ← A[idx]
            idx ← idx + 1
        clauses ← clause

```

While this algorithm isn't the most efficient, it is however readable enough to the point where anyone can understand what it's doing. `SATSolver(...)` checks if the input is a file, and if so, it reads in the file contents, then parses it as a CNF equation. Otherwise, it treats it as a CNF equation and then parses it as one. Then `parse(...)` extracts every numeric token that is not on a comment line, then it uses that numeric data to initialize its variables and data structures. It should be noted that this pseudocode does not handle bad inputs like a negative number of inputs and clauses, as it assumes perfect input. The actual implementation is up to you and me. To see how it was implemented in actual code, you are more than welcome to read the (commented since C++ isn't the most readable at times) source code.

3 Brute Force

The absolute simplest way of solving a SAT problem. In this section, we create our base SAT solver from scratch. We will not add any fancy features such as incremental solving, backtracking, or even basic contradiction detection... at first. In our simple brute force solver, we are given an input as defined above, and we simply solve by checking every possible combination of variable assignment. The pseudocode for this is actually just as simple as it sounds:

```
bool solveCNF():
    for each set of literal assignments:
        bool solved ← true
        for each clause in clauses:
            bool clause_true ← false
            for each literal_assignment in clause:
                clause_true ← clause_true or literal_assignment
            solved ← solved and clause_true
        if solved = true then
            lits ← current_set_of_literals
            return solved
    return false
```

Why this code works should be obvious. We generate up to 2^n sets of literals assignments (where $n = \text{numVars}$) then we check each one against our CNF equation. A clause is true if any one of the literals in the clause evaluates to true, and is false otherwise. And the whole CNF equation is true if every clause has at least one literal whose assignment within the clause is true. In that case, we store that solution, and return true, and in the case where none of the 2^n sets of literals assignments leads to a solvable outcome, we return false. A runtime analysis of this version of the algorithm shows that in the worst case (the equation is unsatisfiable and every combination must be checked) the algorithm runs in $O(2^n ms)$, where 2^n represents the number of combinations we need to check, m is the amount of time it takes to evaluate the whole equation, and s represents the amount of time it takes to generate one of the combinations that we check. This means in the worst case, the algorithm is abysmally slow. But what about the best case? Well, in the best case, our first literal assignment is the solution, so in that case it will run in $\Omega(ms)$. However, the likelihood of the best case ever happening is abysmally low, and the algorithm should be considered to always run in its worst case time.

Once again, you are welcome to read the source code for the brute force solver (with comments).

3.1 Brute Force Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course. To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here: <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>. We gave our current implementation 1000 SAT instances (uf20-91), each with

20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm 2,119,301 milliseconds, which is roughly 35.32 minutes. Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that I'm going to be improving upon anyways.

3.2 Some Improvements